

AI ASSISTED CODING

ASSIGNMENT – 7.2

COURSE : B.TECH\CSE

SPECIALISATION : AIML

NAME : MANDALA.TEJASRI

ENROLLMENT NUMBER : 2403A51171

COURSE : AI ASSISTED CODING

DATE : 16 sep 2025

BATCH NO : 01

Task Description#1:

Use google gemini to correct the syntax errors in conditionals

a = 10

if a = 10:

print("Equal")

PROMPT :

Develop a python code to generate a code without syntax errors

a = 10

if a = 10:

print("Equal")

CODE AND EXPECTED OUTPUT :

```
 a = 10
if a == 10:
    print("Equal")
```

```
 Equal
```

EXPLANATION :

★ Certainly! The code in cell 74de7a25 is a simple Python script that demonstrates a conditional statement.

- `a = 10` : This line assigns the integer value `10` to the variable `a`.
- `if a == 10:` : This is an `if` statement, which checks if the condition `a == 10` is true. The `==` operator is used to compare if the value of `a` is equal to `10`.
- `print("Equal")` : This line is executed only if the condition in the `if` statement is true. In this case, since `a` is equal to `10`, the string "Equal" is printed to the console.

So, the code checks if the variable `a` is equal to `10`, and if it is, it prints "Equal".

Task Description#2

Use google gemini to correct the syntax errors in given code

def sum_upto_n(n):

total = 0

for i in range(1, n):

total += i

return total

PROMPT :

Develop a python code to generate a code without syntax errors

```
def sum_upto_n(n):  
    total = 0  
    for i in range(1, n);  
        total += i  
    return total
```

CODE AND EXPECTED OUTPUT :

```
def sum_upto_n(n):  
    total = 0  
    for i in range(1, n + 1): # corrected range to include n  
        total += i  
    return total  
  
a = 10  
if a == 10:  
    print("Equal")  
  
Equal
```

EXPLANATION :

This Python code defines a function called `sum_upto_n` that calculates the sum of all integers from 1 up to a given number `n`.

- `def sum_upto_n(n):` : This line defines a function named `sum_upto_n` that takes one argument, `n`.
- `total = 0` : This line initializes a variable `total` to `0`. This variable will store the sum of the numbers.
- `for i in range(1, n + 1):` : This is a `for` loop that iterates through a sequence of numbers. `range(1, n + 1)` generates numbers starting from 1 up to and including `n`. In each iteration, the current number is assigned to the variable `i`.
- `total += i` : Inside the loop, this line adds the current value of `i` to the `total`.
- `return total` : After the loop finishes (meaning all numbers from 1 to `n` have been added), this line returns the final value of `total`, which is the sum of all numbers up to `n`.

In essence, the function loops through numbers from 1 to `n` and accumulates their sum, finally returning the accumulated sum.

Task Description#3

Use google gemini to correct the syntax errors in given code

Class user:

```
def __init__(self, name):  
    self.name = name  
  
u = User("Alice")  
print(u.getName())
```

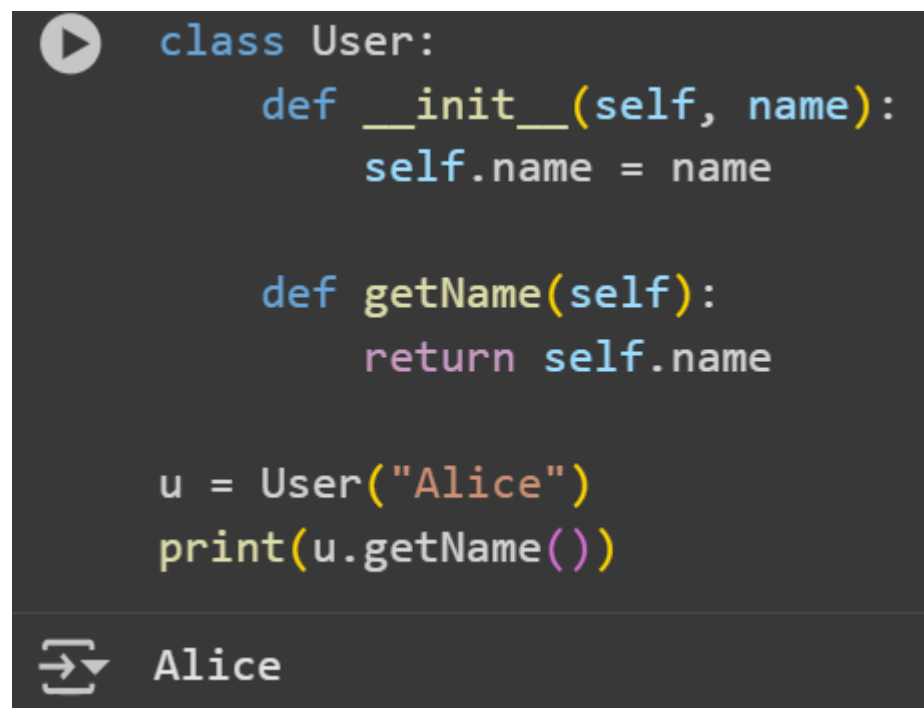
PROMPT :

Develop a python code to generate a code without syntax errors

Class user:

```
def __init__(self, name):  
    self.name = name  
  
u = User("Alice")  
print(u.getName())
```

CODE AND EXPECTED OUTPUT :



The screenshot shows a code editor with a dark background. At the top left, there is a play button icon. The code is written in a syntax-highlighted font. Below the code, there is a separator line, and then the output 'Alice' is displayed next to a right-pointing arrow icon.

```
class User:  
    def __init__(self, name):  
        self.name = name  
  
    def getName(self):  
        return self.name  
  
u = User("Alice")  
print(u.getName())
```

→ Alice

EXPLANATION :

This code defines a simple Python class called `User` and then creates an object (or instance) of that class.

- `class User:` : This line defines a new class named `User` . Classes are blueprints for creating objects.
- `def __init__(self, name):` : This is the constructor method of the `User` class. The `__init__` method is automatically called when you create a new `User` object.
 - `self` : This refers to the instance of the class itself.
 - `name` : This is a parameter that you pass when creating a `User` object.
 - `self.name = name` : This line assigns the value passed in the `name` parameter to an attribute of the object called `name` . Each `User` object will have its own `name` attribute.
- `def getName(self):` : This defines a method within the `User` class called `getName` . Methods are functions that belong to a class and operate on the objects of that class.

- `self` : This refers to the instance of the class itself.
- `name` : This is a parameter that you pass when creating a `User` object.
- `self.name = name` : This line assigns the value passed in the `name` parameter to an attribute of the object called `name` . Each `User` object will have its own `name` attribute.
- `def getName(self):` : This defines a method within the `User` class called `getName` . Methods are functions that belong to a class and operate on the objects of that class.
 - `self` : Again, this refers to the instance of the class.
 - `return self.name` : This method simply returns the value of the `name` attribute for the specific `User` object it's called on.
- `u = User("Alice")` : This line creates a new instance (object) of the `User` class. It calls the `__init__` method and passes the string "Alice" as the `name` argument. This new object is assigned to the variable `u` .
- `print(u.getName())` : This line calls the `getName()` method on the `u` object and prints the value returned by that method. Since `u` was created

Task Description#4

Use google gemini to correct the syntax errors in given code

Class Car:

def start():

print("Car started")

mycar = Car()

mycar.start()

PROMPT :

Develop a python code to generate a code without syntax errors

Class Car:

def start():

print("Car started")

```
mycar = Car()
```

```
mycar.start()
```

CODE AND EXPECTED OUTPUT :

```
class User:
    def __init__(self, name):
        self.name = name

    def getName(self):
        return self.name

u = User("Alice")
print(u.getName())
```

Alice

EXPLANATION :

This code defines a simple Python class called `Car` and then creates an object (or instance) of that class.

- `class Car:` : This line defines a new class named `Car`. Classes serve as templates for creating objects.
- `def start(self):` : This defines a method within the `Car` class called `start`. Methods are functions that are associated with a class and can be called on objects created from that class.
 - `self` : This is a reference to the instance of the class itself. It's a convention to use `self` as the first parameter in instance methods.
 - `print("Car started")` : This line inside the `start` method prints the string "Car started" to the console when the method is called.
- `mycar = Car()` : This line creates a new instance (object) of the `Car` class. This new object is assigned to the variable `mycar`.
- `mycar.start()` : This line calls the `start()` method on the `mycar` object. This will execute the code inside the `start` method, which is `print("Car started")`.

In summary, this code defines a `Car` class with a method to simulate starting a car. It then creates a `Car` object and calls the `start` method on that

Task Description#5

Use google gemini to correct the syntax errors in given code

```
def grade_student(score):
```

```
    If score < 40:
```

```
        return "A"
```

```
    elif score < 70:
```

```
        return "B"
```

```
    else:
```

```
        return "C"
```

PROMPT :

Develop a python code to generate a code without syntax errors

```
def grade_student(score):
```

```
    If score < 40:
```

```
        return "A"
```

```
    elif score < 70:
```

```
        return "B"
```

```
    else:
```

```
        return "C"
```

CODE AND EXPECTED OUTPUT :



```
class User:
    def __init__(self, name):
        self.name = name

    def getName(self):
        return self.name

u = User("Alice")
print(u.getName())
```



Alice

EXPLANATION :

- `def grade_student(score) :` : This line defines a function named `grade_student` that accepts one argument, `score` .
- `if score >= 70 :` : This is the first condition. It checks if the `score` is greater than or equal to 70. If this condition is true, the code inside this `if` block is executed.
- `return "A" :` If the score is 70 or above, the function immediately stops and returns the string "A".
- `elif score >= 40 :` : If the first condition (`score >= 70`) is false, the code moves to this `elif` (else if) condition. It checks if the `score` is greater than or equal to 40. If this condition is true, the code inside this `elif` block is executed.
- `return "B" :` If the score is not 70 or above, but is 40 or above, the function stops and returns the string "B".
- `else :` : If neither of the previous conditions (`score >= 70` or `score >= 40`) is true, the code inside the `else` block is executed. This means the score is less than 40.
- `return "C" :` If the score is less than 40, the function stops and returns the string "C".